Architektúra-refaktor — fejlesztési napló

Az audit megállapításainak javítása · 2026. szeptember

Az 01-es dokumentumban azonosított hibák javítása. Ez a napló azt rögzíti, **mi változott és miért**, hogy a döntések később visszakereshetők legyenek, és a dolgozat megvalósítási fejezete ezekre épülhessen.

1. Indíthatóság és környezet

Probléma. A dokumentált `uvicorn app.main:app` parancs importhibával elszállt, a modell útvonala pedig abszolút konténer-útvonal volt.

Megoldás. Minden belső import `app.` prefixű abszolút import lett, a csomagok `__init__.py`-t kaptak. A modell útvonala a `MODEL_PATH` környezeti változóból jön, alapértelmezésben a fájl melletti relatív útvonalról. A konténer `WORKDIR`-je `/srv`, így ugyanaz a parancs fut lokálisan és Dockerben.

Ellenőrzés. `python3 -c "import app.main"` a repó gyökeréből hibátlanul lefut; a detektor a `docker` konténeren kívül is betölti a modellt.

2. Konfiguráció és titkok

Minden beállítás a `app/config.py`-ba került, környezeti változókból olvasva. A `.env.example` verziókezelt sablon, a `.env` gitignore-olt és generált jelszót tartalmaz. A Compose fájlok kizárólag a `.env`-ből veszik a jelszót, és hiányzó érték esetén indításkor hibát dobnak, nem csendben alapértelmezésre esnek.

A függőségek verzióhoz rögzítve (`docker/requirements-api.txt`), így a build reprodukálható — kutatási artefaktumnál ez alapkövetelmény.

Compose fájlok a repó gyökerében. A Compose a projekt könyvtárából olvassa a `.env`-et, ami a compose fájl mappája — a `docker/` alatti elhelyezés miatt a `${POSTGRES_PASSWORD}` behelyettesítés nem oldódott volna fel. A `compose.yaml` és `compose.prod.yaml` ezért a gyökérbe került; a Dockerfile-ok és Nginx konfigok maradtak a `docker/` mappában. A `.dockerignore` kizárja a `.env`-et, így titok nem kerülhet image-be.

Fejlesztői reverse proxy. A dev stack is Nginx mögött fut, ugyanazon a `/api` és `/ws` útvonalon, mint a production. A webpack hot-reload socketje `/ws-hmr`-re került, hogy ne ütközzön a gesztus-WebSockettel.

3. Gesztusfelismerés

`IMAGE` → `VIDEO` **mód.** A MediaPipe HandLandmarker mostantól `RunningMode.VIDEO`-ban fut, monoton növekvő időbélyegekkel. Így a modell képkockák között követi a kezet ahelyett, hogy minden képkockán újrafuttatná a teljes tenyérdetektálást.

Kliensenkénti példány. A korábbi egyetlen, globális detektor helyett minden munkamenet saját `HandGestureDetector` példányt kap. Ezzel megszűnt az az osztály hiba, hogy két egyidejű felhasználó felülírta egymás swipe-bázisértékét. A `SessionManager` kezeli a példányok életciklusát, korlátozza az egyidejű munkamenetek számát és felszabadítja a tétlen munkameneteket.

Távolságfüggetlen swipe. Az elmozdulás mostantól a kéz méretéhez (csukló–középső ujj tő távolság) normalizálva számítódik, nem nyers normalizált koordinátában. Ugyanaz a fizikai mozdulat 50 cm-ről és 2 m-ről is ugyanúgy vált ki gesztust.

Lézermutató. A mutatóujj pozíciója külön üzenettípusban, normalizált (0–1) koordinátaként érkezik — nem a gesztus szövegébe kódolva, ahogy korábban. Adaptív holtsávos exponenciális simítás tartja stabilan a mutatót álló kéznél, miközben mozgás közben követő marad.

Statikus gesztusok. A `Zoom In` / `Zoom Out` osztályozás ujj-kinyújtottsági feltételt is kapott a szögküszöbök mellé, mert önmagában a szög egy ellazult kéznél is tévesen aktiválódhat. A küszöbök `.env`-ből hangolhatók, a `tools/gesture_probe.py` pedig élőben mutatja a saját kézre mért szögeket és CSV-be rögzíti őket — ez adja a küszöbválasztás dokumentálható módszertanát.

4. Átvitel: csak WebSocket

A `POST /process_frame` végpont megszűnt. A képkockák a WebSocketen mennek, 4 bájtos sorszám-fejléccel, ami két dolgot old meg egyszerre: a kliens párosítani tudja a válaszokat a képkockákhoz, és **óraszinkron nélkül** tudja mérni a körbeérési időt.

A frontenden egyetlen `useGestureStream` hook végzi a teljes átvitelt mindkét oldalon, egységesen:

- egyetlen, újrahasznált canvas (korábban minden ciklusban új canvas elem készült),
- 640 px-re skálázott, 0,7 minőségű JPEG (korábban teljes felbontás, ~0,92 minőség),
- visszanyomás-kezelés a socket pufferének figyelésével,
- automatikus újracsatlakozás exponenciális várakozással,
- `onerror` / `onclose` kezelés és látható állapotjelzés.

5. Biztonság

Terület	Korábban	Most
Identitás	kliens által választott `clientId`	szerver által kiadott, opak token
WebSocket	bárki csatlakozhatott más azonosítójával	csak érvényes tokennel, `4401` elutasítás
CORS	* + `allow_credentials=True`	env-ből állítható lista; wildcard esetén a credentials automatikusan tiltott
Terhelés	korlátlan	kliensenkénti képkocka-korlát, REST rate limit, munkamenet-limit
Bemenet	méretellenőrzés nélkül	maximális képkocka-méret, hibaüzenet a kapcsolat bontása nélkül
Titkok	Compose-ba írt jelszó	`.env`, generált jelszóval, verziókezelésből kizárva
Konténer	root felhasználó	dedikált, jogosultság nélküli felhasználó + healthcheck

6. Adatbázis és telemetria

Az adatbázis-réteg élesítve. A séma négy új táblával bővült (`gesture_events`, `frame_samples`, `practice_attempts`, `session_metrics`), a hozzájárulás pedig ténylegesen rögzítésre kerül, verziószámmal és időbélyeggel.

A telemetria-írás **sosem lassítja a gesztus-pipeline-t**: a sorok egy memóriában tartott sorba kerülnek, és egy háttérfeladat írja ki őket kötegelve. Ha a sor megtelik, a mérési sor eldobódik — a működés fontosabb, mint a mérés.

A képkockák továbbra sem kerülnek tárolásra, csak metaadat.

7. Frontend hibák

A testreszabott leképezés nyelvi hibája. A beállítások mostantól a backend kanonikus gesztusnevére vannak kulcsolva, és az értékek is kanonikus akció-azonosítók (`nextSlide`, `hidePerson`, …), nem lefordított szövegek. A megjelenített felirat csak címke. Ugyanez a hiba a gyakorló felületen is javítva: a helyes/helytelen visszajelzés a kanonikus névvel hasonlít össze.

Gesztuskészlet. A felület a backendtől kapott listából épül fel, így nem kínál olyan gesztust gyakorlásra, amit a rendszer nem tud visszaigazolni. Új gesztus bevezetésekor a felület automatikusan bővül.

Kamerahiba. A `NotAllowedError` / `NotReadableError` és az egyéb esetek megkülönböztetve, magyarul, románul és angolul jelennek meg a felhasználónak.

MediaPipe assetek. A szegmentáló WASM és modell fájlok a build részeként, saját origin-ről szolgálódnak ki, nem publikus CDN-ről.

8. Takarítás

Törölve: `app/fps.py`, `app/api/process.py`, `app/models/pose_detectation.py`, `app/client/*.py`, valamint a frontend árva fájljai és tíz használaton kívüli npm csomag.

Megtartva a fejlesztő kérésére: `app/models/segmentation.py`, az `upsert_client` (immár ténylegesen használva), és a `browser_extension/`.

9. Gesztuskészlet bővítése

Orientációfüggetlen ujjérzékelés. A korábbi logika a kép y- és x-tengelyéhez mérte az ujjak kinyújtottságát, ami csak egy adott kéztartásnál működött megbízhatóan. Az új megoldás a csukló→ujjtő vektort veszi a kéz saját tengelyének, és arra vetíti az ízületeket — így ugyanaz a döntés születik felfelé tartott és oldalra fordított kéznél is.

`Option 1–5` **visszaépítve.** A számlálás a kinyújtott ujjak számából áll elő, és csak oldalra fordított kéznél aktív. Ez választja el a számlálást a függőleges zoom-pózoktól: egy felfelé tartott nyitott tenyér egyébként megkülönböztethetetlen lenne az „Option 4"-től. A rendszer emellett megköveteli, hogy a felemelt ujjak a mutatóujjtól kezdve összefüggőek legyenek — a szórt minta jellemzően félbehagyott kéztartás, nem szándékos számlálás.

Zoom a személyre. A `Zoom In` / `Zoom Out` a kivágott előadó méretét állítja a dián (0,6–1,8× között), ahogy a dolgozat leírja. A kompozitálás külön vászonra került, hogy a nagyítás csak az előadót érintse, a hátteret és a diát ne. Alapértelmezetten 1,0× — a megjelenés változatlan, amíg a felhasználó nem zoomol.

A felület a backend gesztuslistájából épül fel, így az új gesztusok külön frontend-módosítás nélkül jelentek meg a gyakorló és a beállítások oldalon.

10. A gesztusfolyam útvonala: `/api/ws`

A fejlesztői szerver (webpack-dev-server) saját hot-reload WebSocketje alapértelmezés szerint a `/ws` útvonalat foglalja, és elkapta a gesztus-kapcsolatot, mielőtt a proxy továbbíthatta volna a backendnek. A böngésző így a fejlesztői szerverrel beszélgetett: a képkockákat elküldte, de gesztuseseményt soha nem érkezett vissza.

A `WDS_SOCKET_PATH` beállítás erre nem megoldás: a react-scripts csak a kliensoldali URL-t állítja vele, a szerveroldali socket marad a `/ws`-en. A gesztusfolyam ezért az `/api/ws` útvonalra került. Ez egyben tisztább is: a teljes backend — REST és WebSocket — egyetlen prefix alatt van, így fejlesztői proxyban, Nginxben és Caddyben is egyetlen szabály fedi le.

11. Ami még hátravan

1. **Küszöbkalibráció valós kézzel** a `tools/gesture_probe.py` eszközzel — a jelenlegi értékek irodalmi/megfigyelési alapon vannak beállítva, nem a végleges felhasználó kezére.
2. A `browser_extension` bekötése a valós API-hoz (jelenleg egy helyi stub bridge-et pollingol).
3. Terheléses mérés több egyidejű klienssel.
4. Éles telepítés: domain, TLS, `ALLOWED_ORIGINS` szűkítése.